

Deep Dive into Android Security



Ahmethan Gültekin
#raconf2026





Ahmethan Gültekin

mobile application security researcher
founder of **Byteria**



ahmeth4n.github.io



byterialab.com

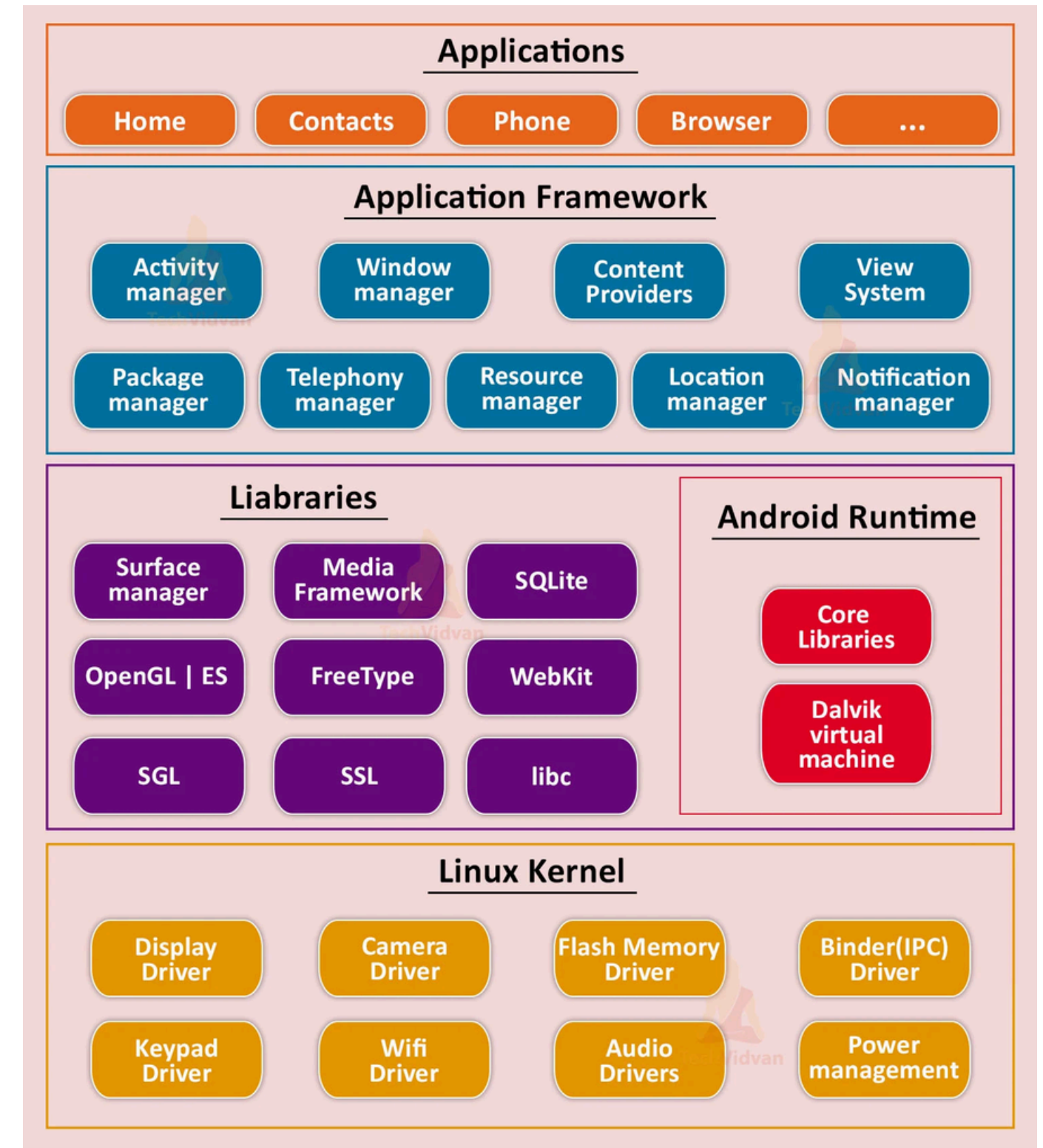
Agenda

- Android Fundamentals
- Native Internals
- Use of RE Tools (r2,ghidra,frida etc..)
- Practice
- RASP protection techniques
- Real-World App Analysis (keep it a secret)
- whatever else we feel like talking about

Android Fundamentals

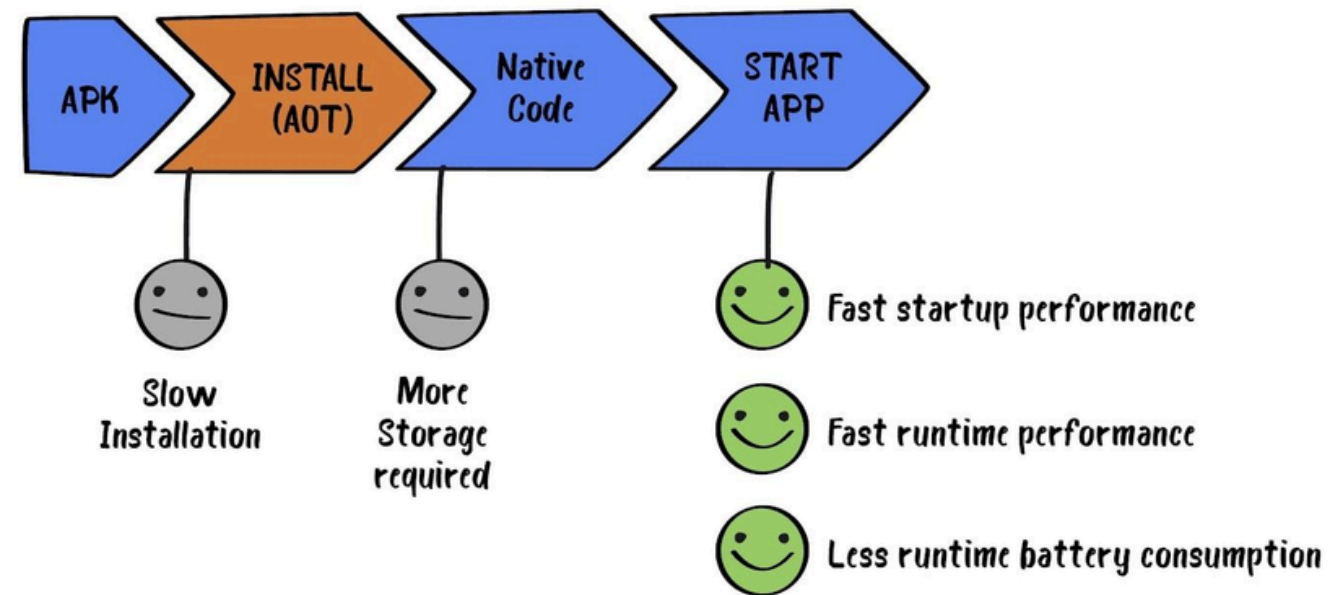
Linux çekirdeği üzerine geliştirilmiş
APK paket formatını kullanan
Kendine ait *sandboxing - security policyleri* olan
Geniş ve büyük ekosisteme sahip bir işletim sistemidir.

Android (> 4.4) - ART compilation (new) / JIT
Android (< 4.4) - DVM compilation (old) / AOT



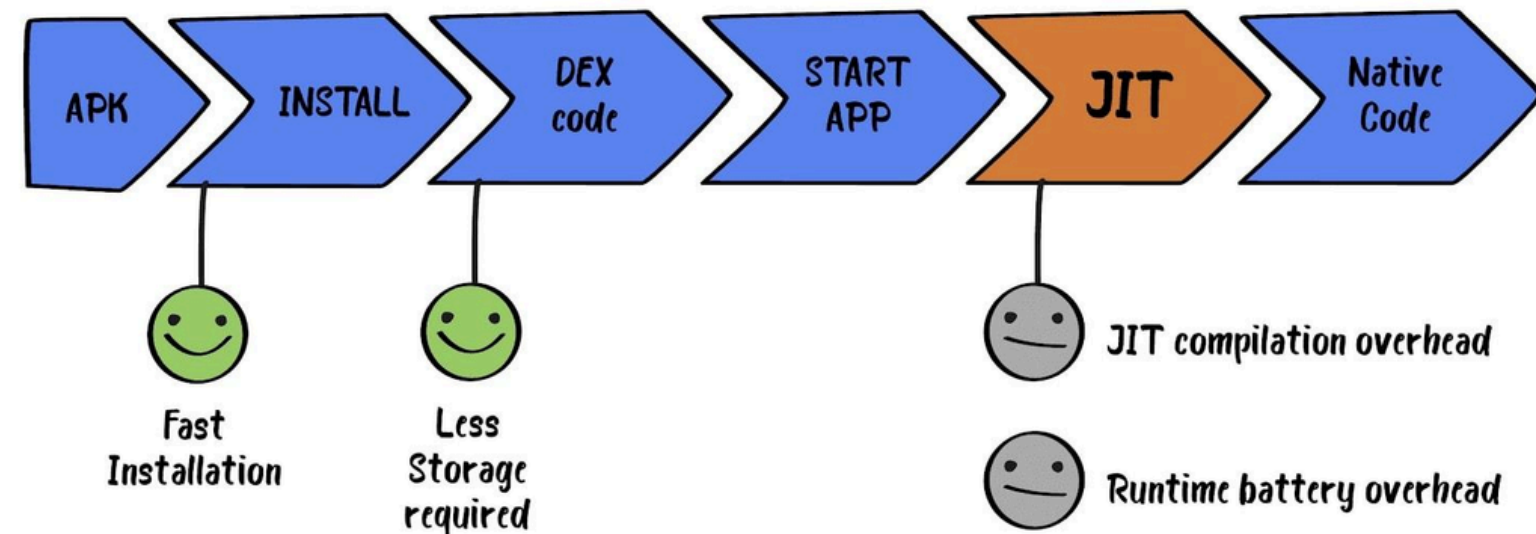
Android Fundamentals

Android Marshmallow Ahead-Of-Time (AOT) Solution



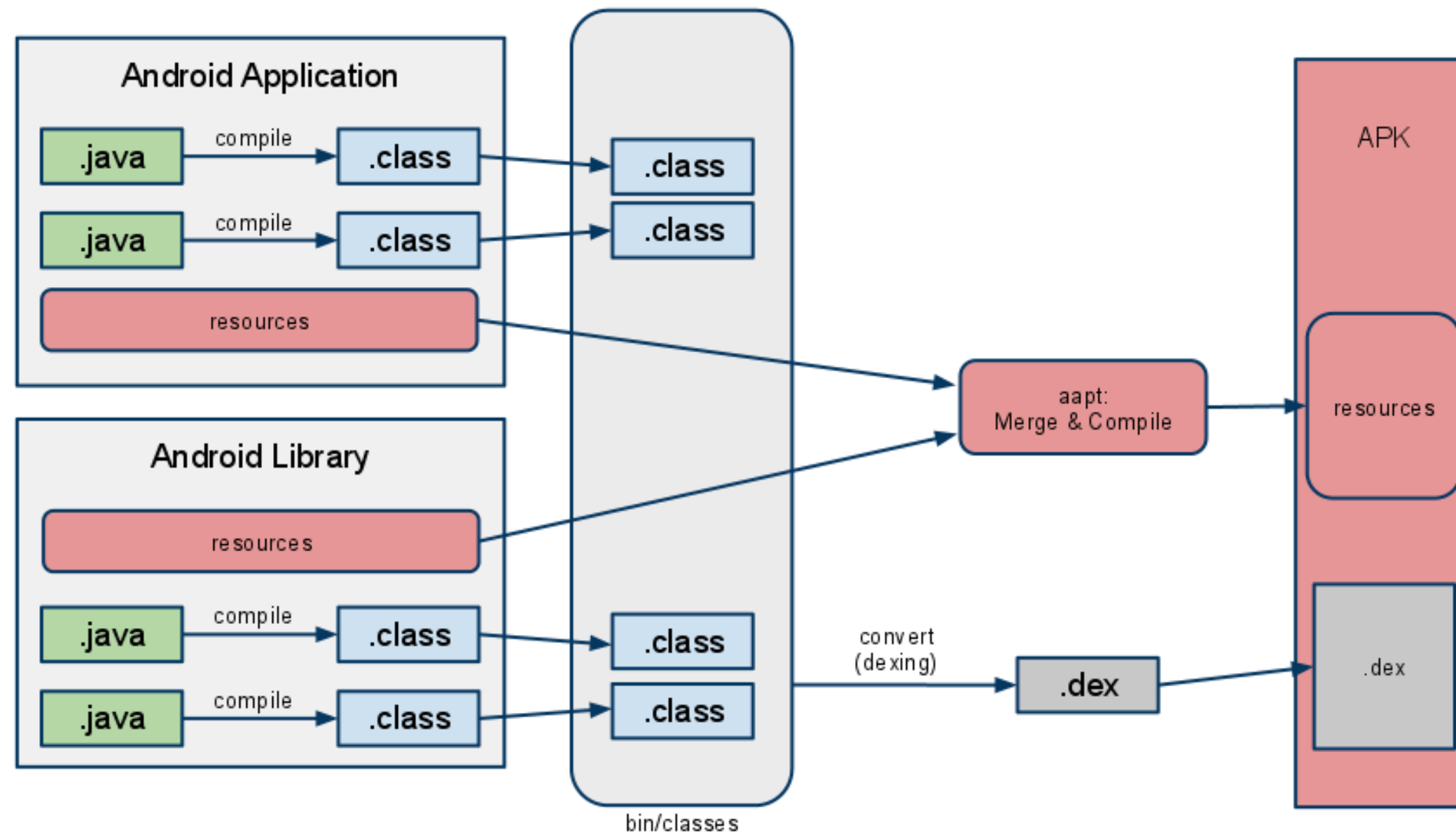
Yeni Android sürümlerinin (ART) kullandığı

Android KitKat Just-In-Time (JIT) Solution



Eski Android sürümlerinin (DVM) kullandığı

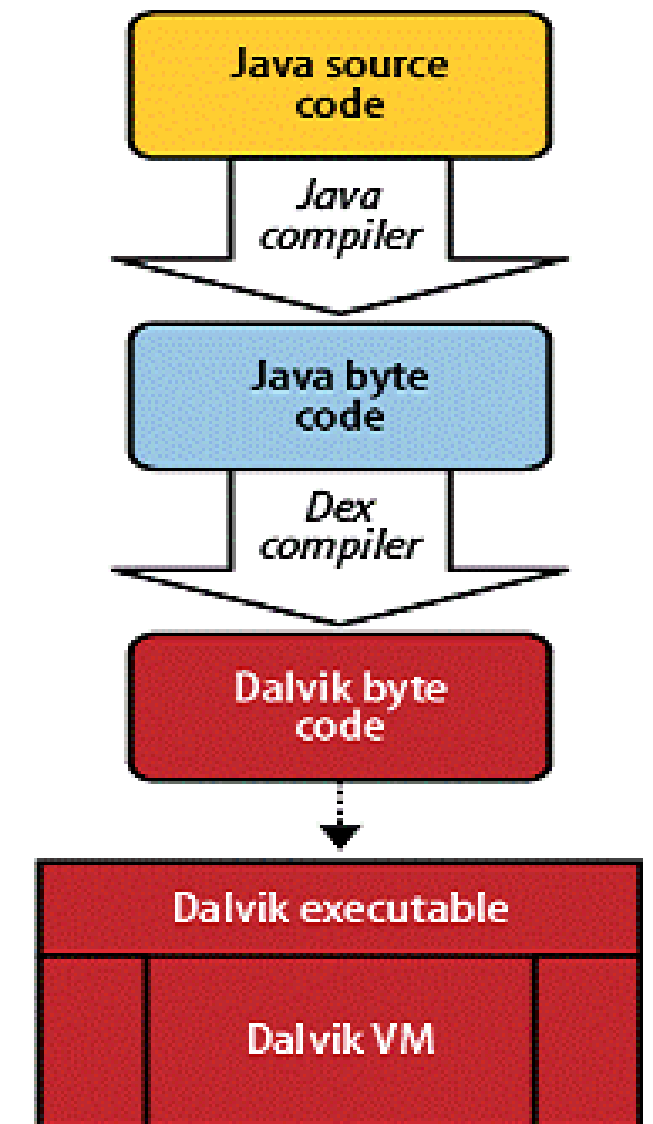
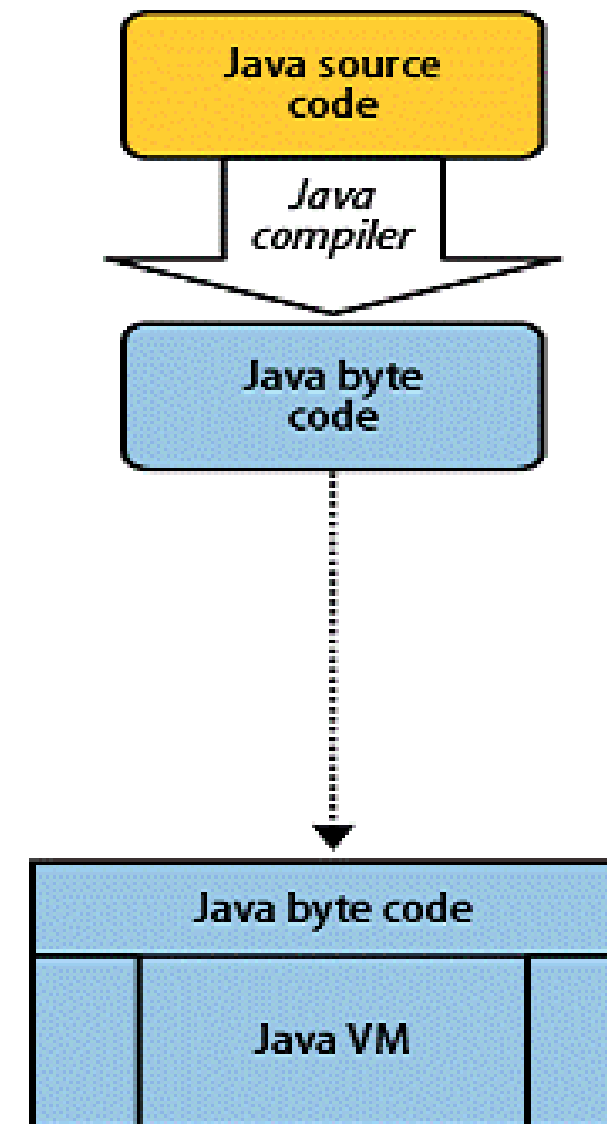
Android Fundamentals



.dex → Dalvik Executable File
aapt: Android Asset Packaging Tool

Android Fundamentals (DEX)

- Android uygulamalarının çalıştırılabilir kod formatıdır (.dex)
- Java/Kotlin bytecode → DEX'e dönüştürülür
- Android Runtime (Android Runtime / ART) tarafından çalıştırılır
- Tek bir DEX içinde birden fazla class bulunabilir
- Optimize edilmiştir → daha az bellek ve daha hızlı çalışma

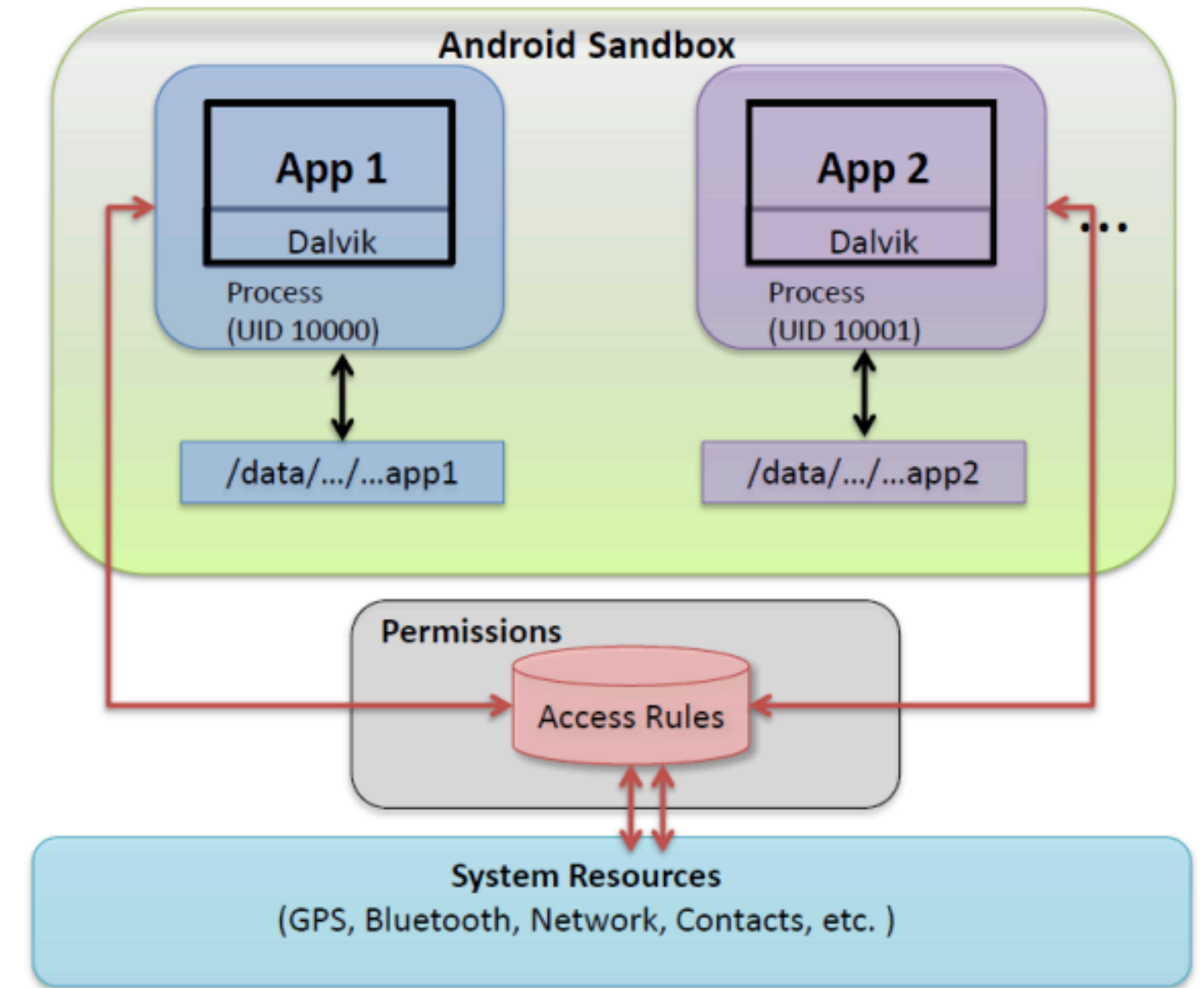


Android Security Model

Sandboxing
SELinux
Permissions
App Signing
Verified Boot

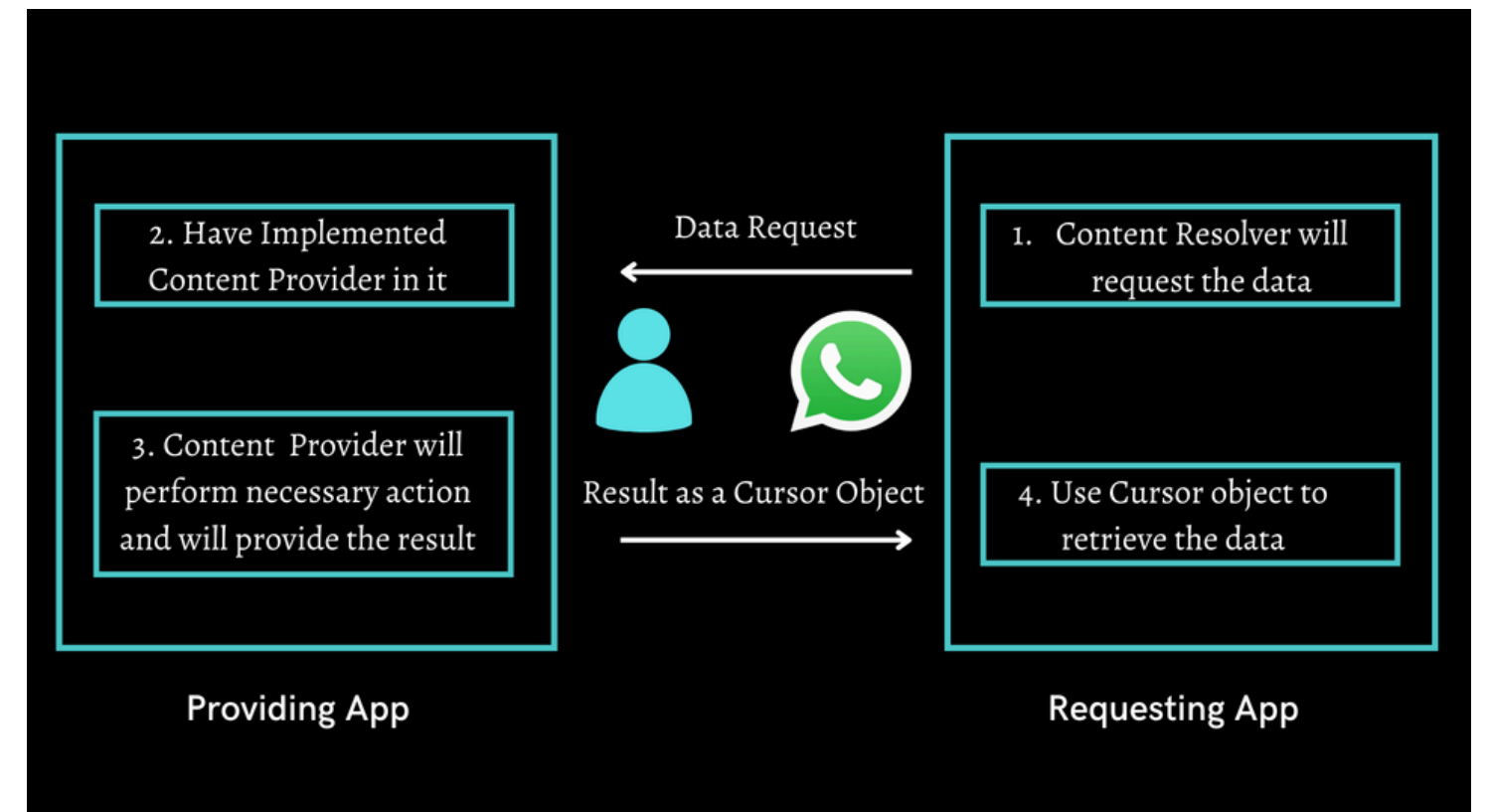
Android Security Model - Sandbox

- Her app kendine ait özgü UID değerine sahiptir
- Bir uygulama başka bir uygulamanın verilerine erişemez
- Her uygulama ayrı bir process (süreç) içinde çalışır
- Uygulamalar arası erişim izinler (permissions) ile kontrol edilir
- Veri paylaşımı sadece **Content Provider / IPC** mekanizmaları ile yapılır
- Uygulamanın dosyaları kendi private dizininde saklanır (/data/data/...)



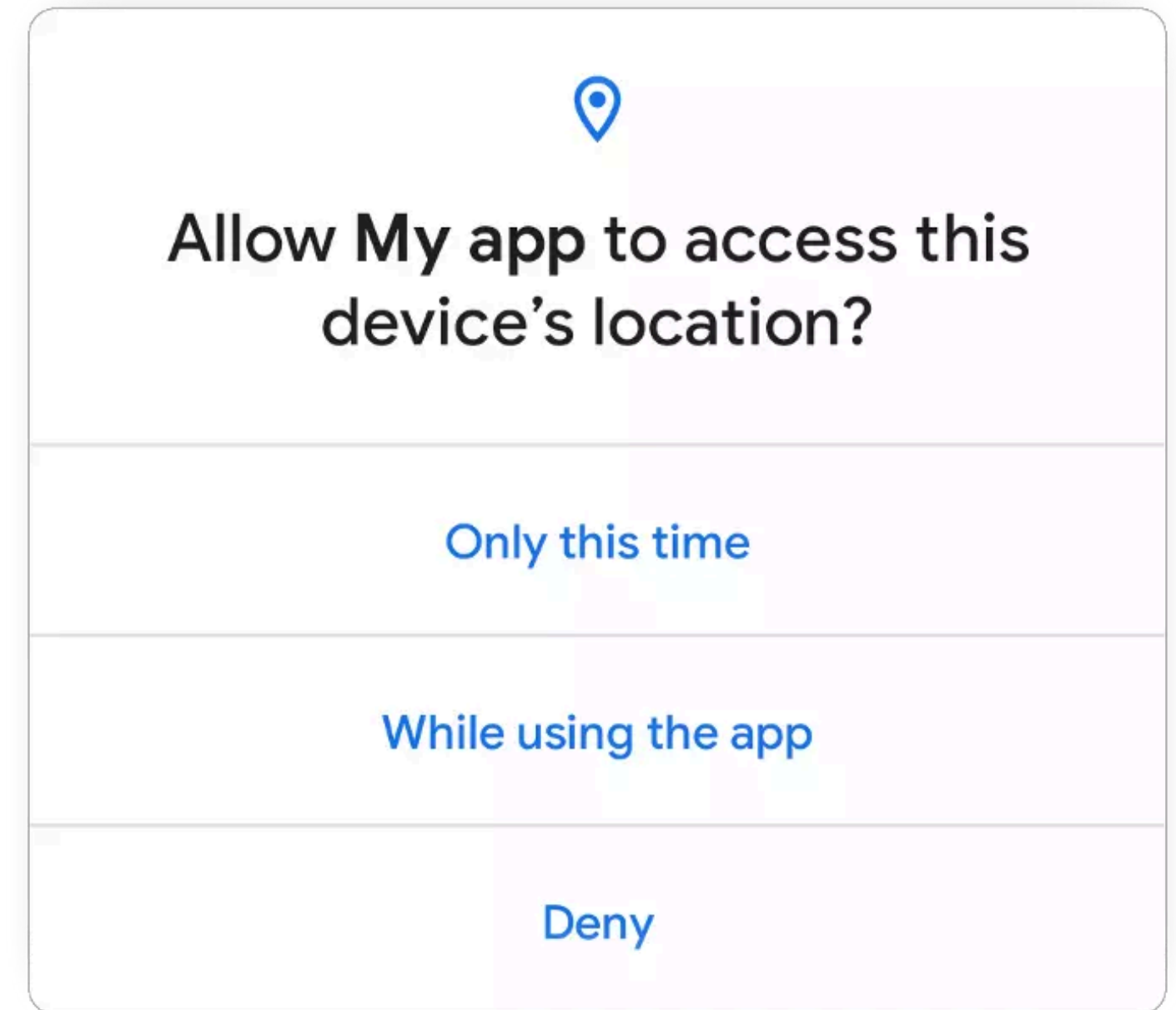
Android Security Model - Content Provider / IPC

- Bir uygulama başka bir uygulamadan veri talep eder
- Veri talebi Content Resolver üzerinden yapılır
- Veri sağlayan uygulama Content Provider içerir
- Content Provider isteği işler ve gerekli veriyi hazırlar
- Veri kontrollü ve izinli şekilde paylaşılır
- Bu veri paylaşımı IPC (Binder) üzerinden gerçekleştirilir



Android Security Model - Permission

- Uygulamalar hassas sistem kaynaklarına doğrudan erişemez
- Erişim için kullanıcıdan açık izin (permission) alınır
- İzinler çalışma zamanında (runtime) kullanıcıya sorulur
- Kullanıcı izinleri istediği zaman reddedebilir veya geri alabilir
- Her izin belirli bir kaynağa erişimi temsil eder (kamera, konum vb.)
- İzinler uygulamanın erişim yetkisini sınırlar

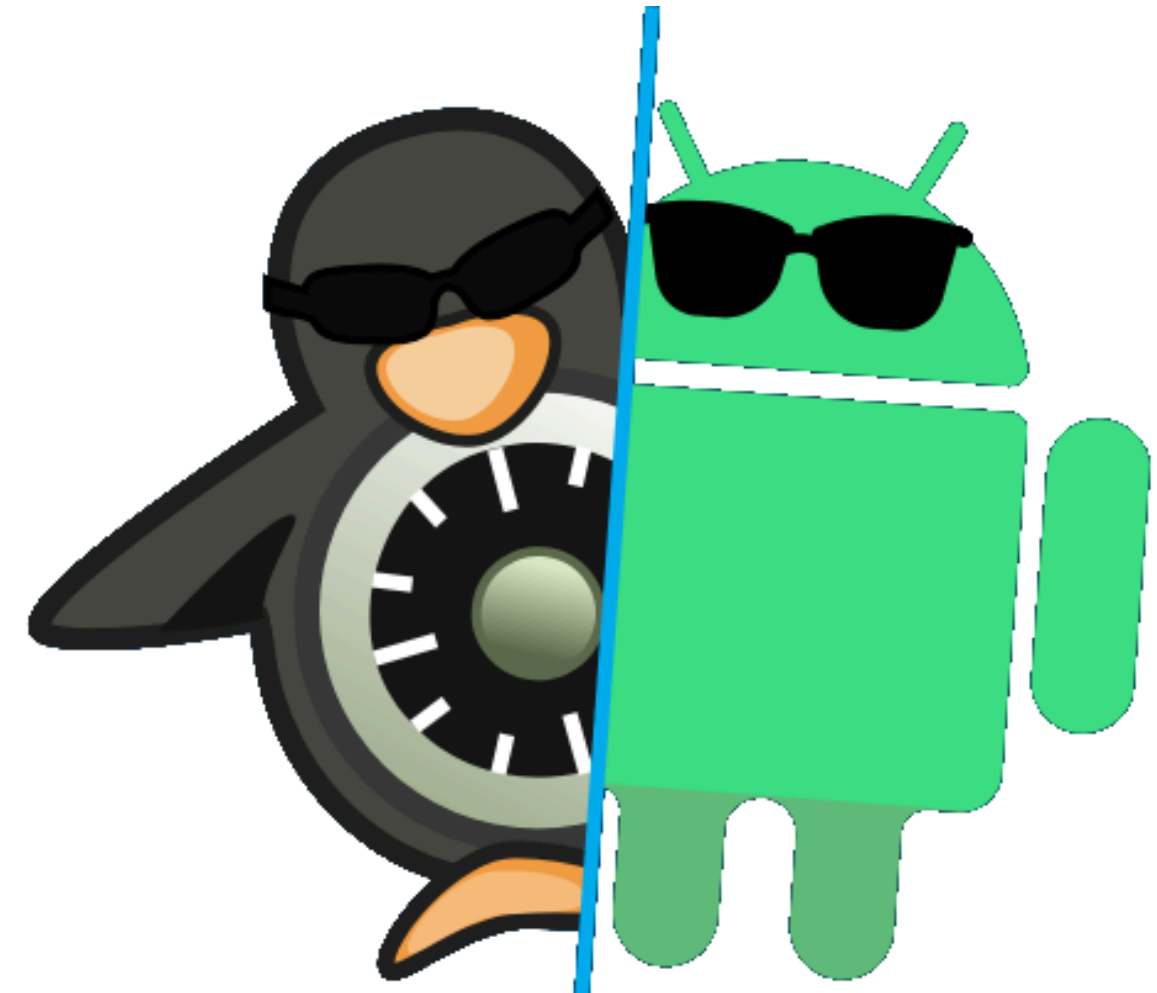


Android Security Model - SELinux

- the worst one
- SELinux, Android'de ek bir güvenlik katmanıdır
- Sistem seviyesinde zorunlu erişim kontrolü uygular
- Tüm erişimler policy (kurallar) ile sınırlandırılır
- İzin verilmiş olsa bile bazı işlemleri engelleyebilir
- Sistem güvenliğini ek bir katman olarak güçlendirir
- Permissive Mode - Enforcing mode olarak 2 moda sahiptir.

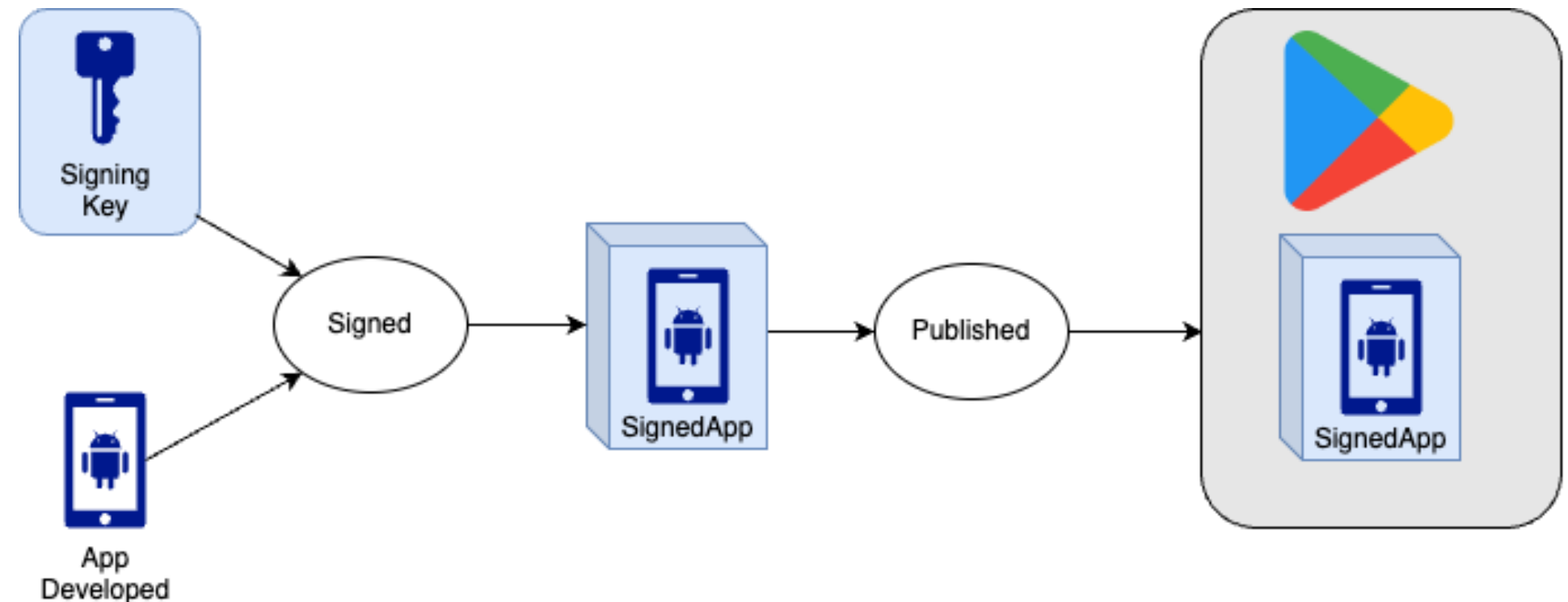
Permissive: Politikalar loglanır, ancak hiçbir şey engellenmez.

Enforcing: SELinux politikalarını katı bir şekilde uygular ve izin verilmeyen işlemleri engeller



Android Security Model - Signing

- Her uygulama developer key ile imzalanır
- İmza → uygulamanın kimliği + bütünlüğü
- Sistem, yükleme sırasında imzayı doğrular
- APK değiştirilirse → imza bozular, yüklenmez
- Güncellemeler sadece aynı key ile yapılabilir
- Aynı key → uygulamalar arası güvenli paylaşım (signature permission)



Android Code Level

Smali side
JNI/Native side

Android Code Level - Smali

- Android'in düşük seviyeli assembly benzeri dilidir
- DEX bytecode'un okunabilir hali
- Register tabanlıdır (v0, v1 vs.)
- İnsan tarafından düzenlenebilir

```
public class Calculation {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

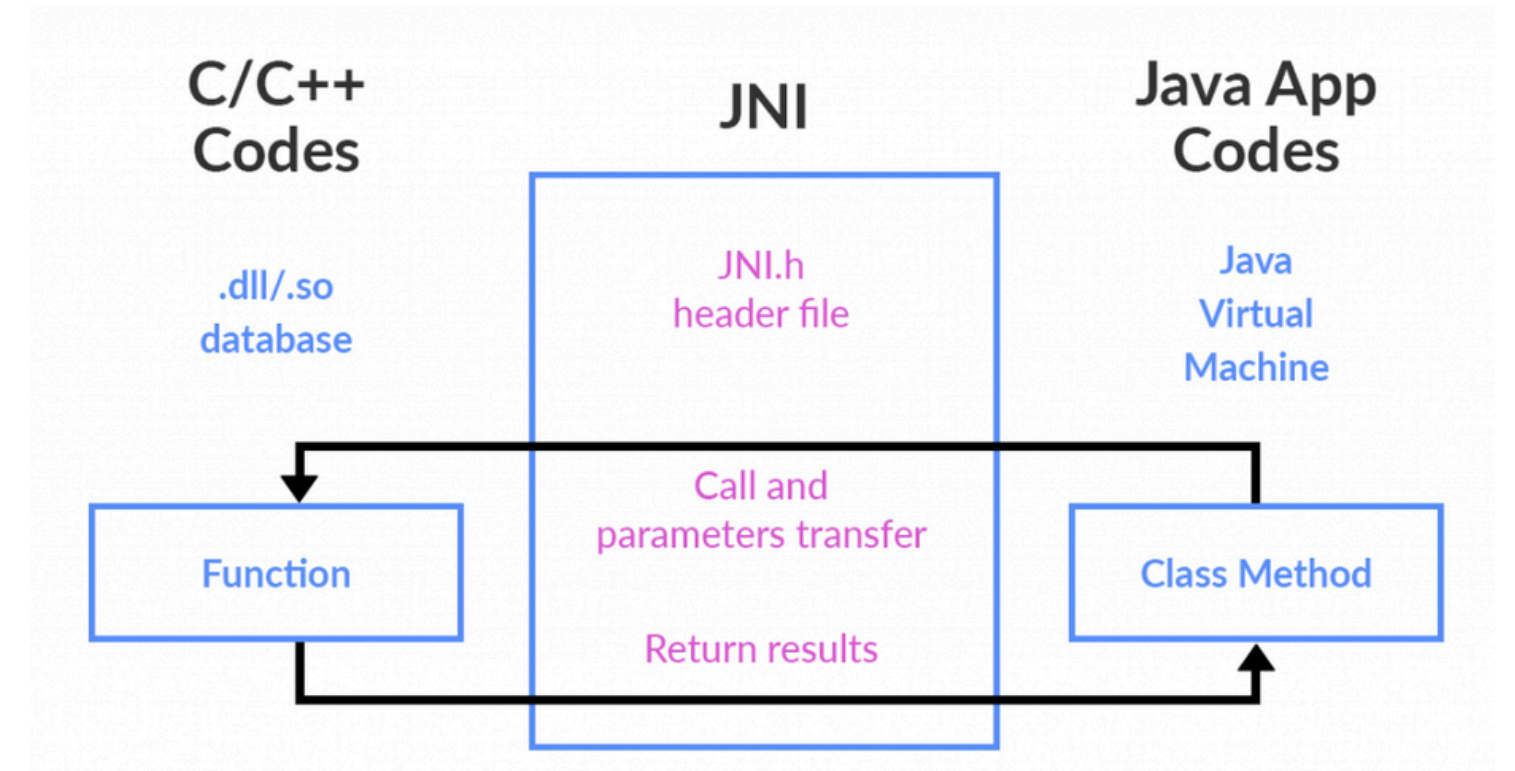
Java code

```
.class public LCalculation;  
.super java/lang/Object  
  
.method public constructor <init>()V  
    .registers 1  
    invoke-direct {p0}, Ljava/lang/Object;-><init>()V  
    return  
.end method  
  
.method public add(II)I  
    .registers 3  
    .parameter "a"  
    .parameter "b"  
  
    .prologue  
    .line 4  
    add-int/2addr v0, v1  
    return v0  
.end method
```

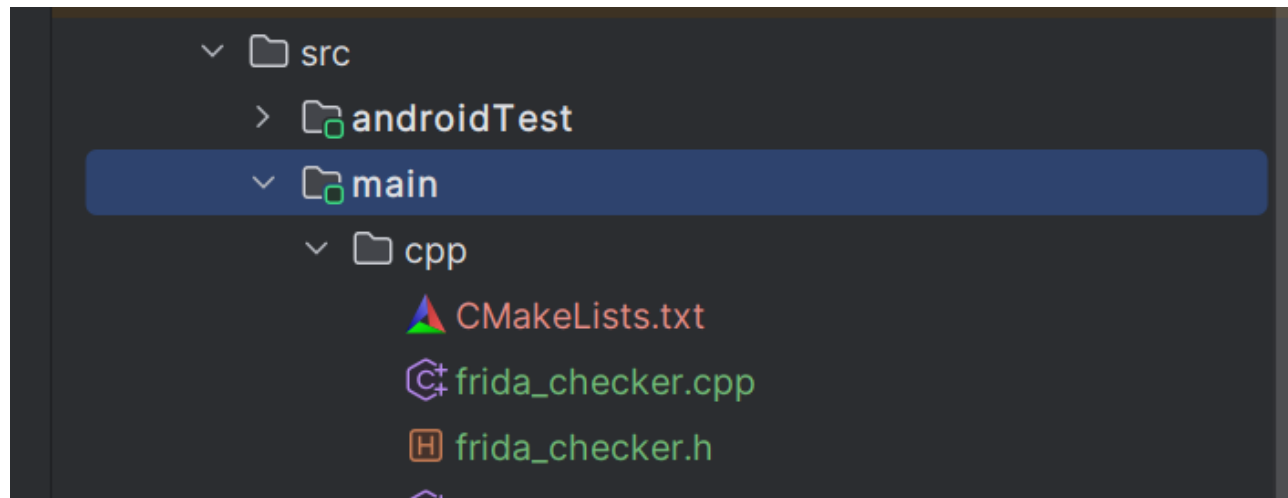
Smali code

Android Code Level - JNI

- Java/Kotlin kodunun C/C++ (native) kod ile konuşmasını sağlar
- Android'de performans kritik veya düşük seviye işlemler için kullanılır
- Native kodlar genelde .so (shared library) olarak gelir
- Gizli kalması istenen kod parçaları native tarafta geliştirilip obfuscation yöntemiyle statik analizden gizli tutulabilir.



Android Code Level - JNI



- **CMakeLists.txt**: NDK projenizi derlemek için kullanacağınız cmake config dosyası
- **.h dosyaları**: C/C++ dillerinde kullanılan başlık dosyaları
- **.cpp dosyaları**: Modüllerin bulunduğu cpp dosyası

Android Code Level - JNI

Java primitive type	Native primitive type
void	void
byte	jbyte
int	jint
float	jfloat
double	jdouble
char	jchar
long	jlong
short	jshort
boolean	jboolean

Android Code Level - JNI

```
extern "C" JNIEXPORT jstring JNICALL
Java_com_example_translatenativemethod_MainActivity_stringFromJNI(
    JNIEnv* env,
    jobject MainActivity /* this */) {
    std::string hello = "Hello from C++";
    return env->NewStringUTF( bytes: hello.c_str());
}
```

```
1 package com.raconf.lab01;
2
3 /* JADX INFO: loaded from: classes3.dex */
4 public class NativeRootDetector {
5     public static native void crashAbort();
6
7     public static native boolean isRooted();
8
9     static {
10         System.loadLibrary("rootdetector");
11     }
12 }
```

- **JNIEXPORT:** Fonksiyonun .so içinden export edilmesini sağlar (JNI tarafından erişilebilir olur)
- **jstring:** JNI'nin Java String karşılığı veri tipi
- **JNICALL:** JNI'nin doğru calling convention ile çağırabilmesini sağlar
- **extern "C":** name manglingi engelleyerek fonksiyon isminin aynı kalmasını sağlar

Android Security Model

Reversing Tools / Usages
(some fun facts)

Reversing Tools - apktool

- Apktool açık kaynaklı bir Android uygulama analiz aracıdır
- APK dosyalarını decode ederek smali ve kaynak dosyalarına ayırır
- yeniden derleme (rebuild) imkânı sağlar
- AndroidManifest ve resource dosyalarını düzenlemeye olanak tanır
- statik analiz ve modifikasyon için kullanılır

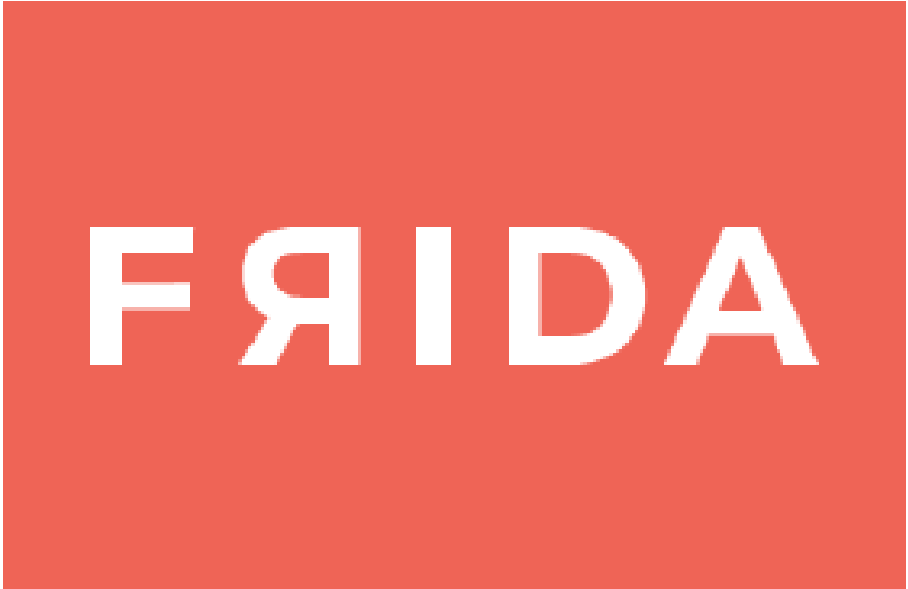


Reversing Tools - Jadx

- JADX açık kaynaklı bir Android decompiler aracıdır
- APK ve DEX dosyalarını Java kaynak koduna çevirir
- statik analiz için kullanılır
- GUI ve CLI versiyonları bulunur
- Android uygulama analizi için yaygın olarak kullanılır
- Gerçekle 1:1 aynı olmasa da gerçeğe yakın seviyede kod üretmeye olanak sağlar.



Android Security Model

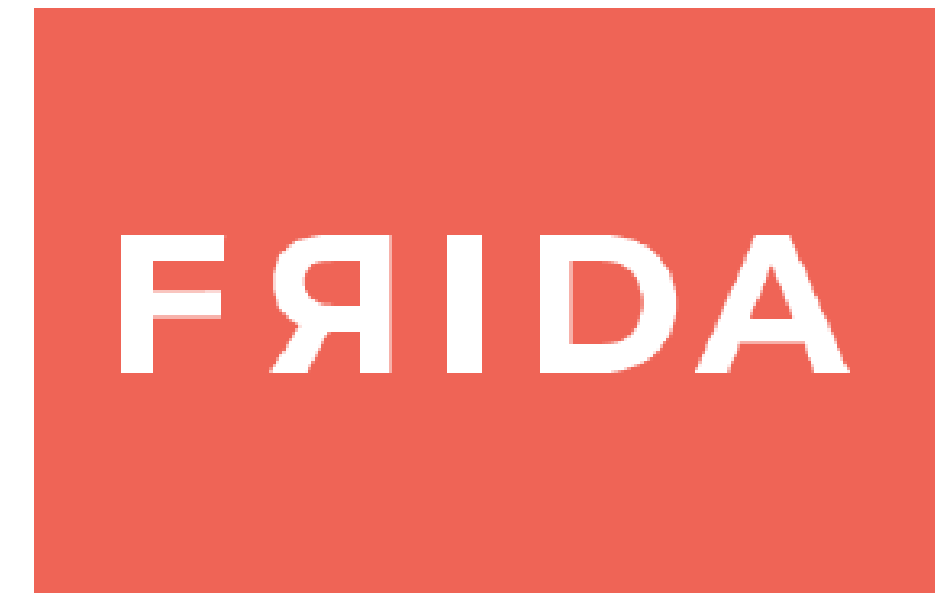


FRIDA

- Frida Agent
- Frida Gadget
- Frida Trace

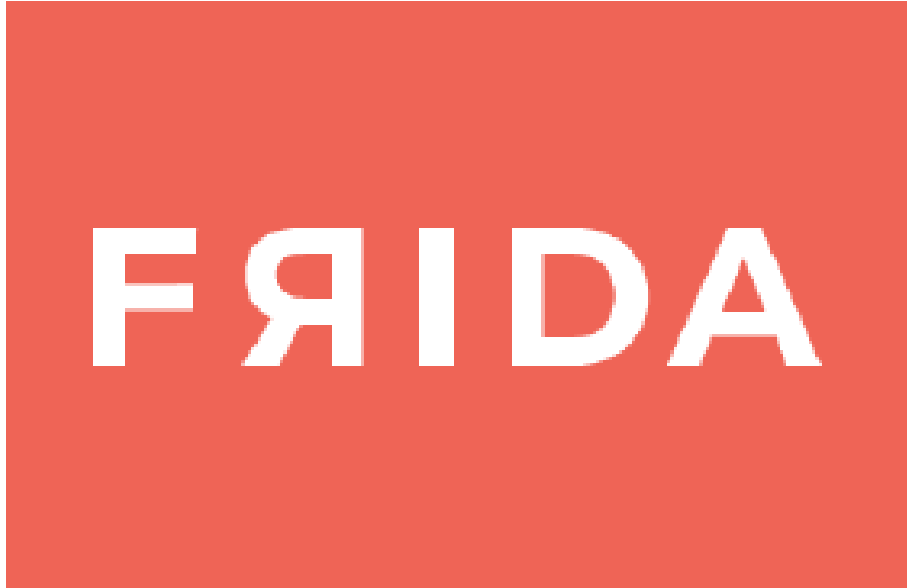
Reversing Tools - Frida

- Frida: dynamic instrumentation için kullanılan bir araçtır
- C ve Vala dili ile geliştirilmiştir
- cihazla etkileşim için genellikle frida-server çalıştırılması gerekir
- USB veya TCP/IP üzerinden iletişim sağlar
- hedef prosese frida-agent enjekte ederek çalışır
- Android'de ptrace() tabanlı attach/spawn yöntemleri ile injection yapılır
- javascript dili ile custom hook scriptleri yazılabilir
- Java ve native (JNI) katmanında hooklama yapılabilir



Reversing Tools - Frida (ptrace)

- ptrace(): bir process'in başka bir process'i kontrol etmesini sağlar
- PTRACE_ATTACH ile hedef prosese bağlanılır ve process durdurulur
- bellek / register üzerinde işlem yapılır
- dlopen() ile shared library yüklenir
- process tekrar çalıştırılır (PTRACE_DETACH)

The logo for Frida, featuring the word "FRIDA" in white, bold, uppercase letters on a solid red rectangular background.

FRIDA

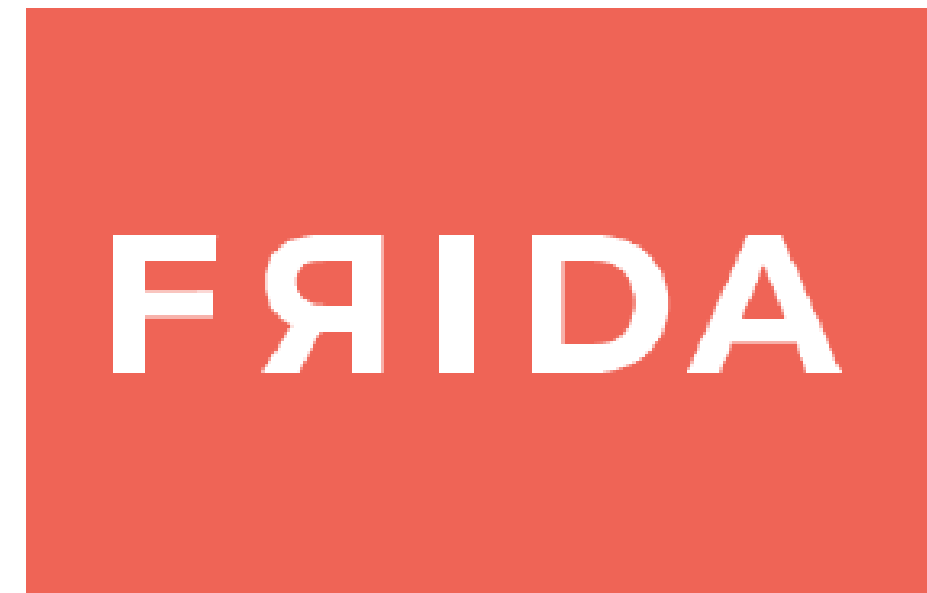
Reversing Tools - Frida (ptrace)

- frida → ptrace(PTRACE_ATTACH)
- → target process (stopped)
- → memory / register manipulation
- → remote dlopen("libfrida-agent.so")
- → agent loaded
- → ptrace(PTRACE_DETACH)
- → target process (running)



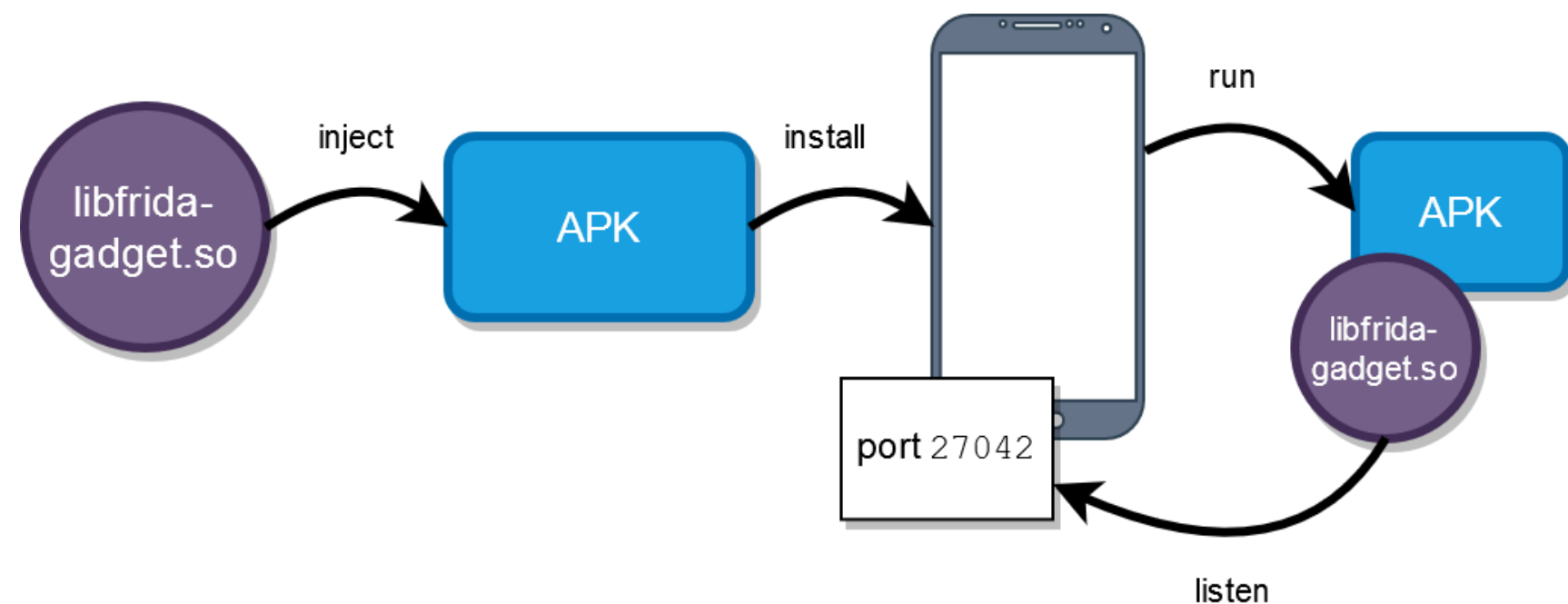
Reversing Tools - Frida API

- **Interceptor:** native fonksiyonlara hook atma (onEnter / onLeave)
- **Module:** yüklü kütüphaneleri bulma, export/symbol arama
- **Memory:** bellek okuma, yazma ve tarama işlemleri
- **NativeFunction:** native fonksiyonları JS içinden çağırma
- **NativeCallback:** JS fonksiyonunu native fonksiyon gibi kullanma
- **Java:** Android Java katmanına erişim ve hooklama
- **Thread:** thread bilgilerini alma ve yönetme
- **Process:** süreç (PID, modüller, bellek yapısı) hakkında bilgi



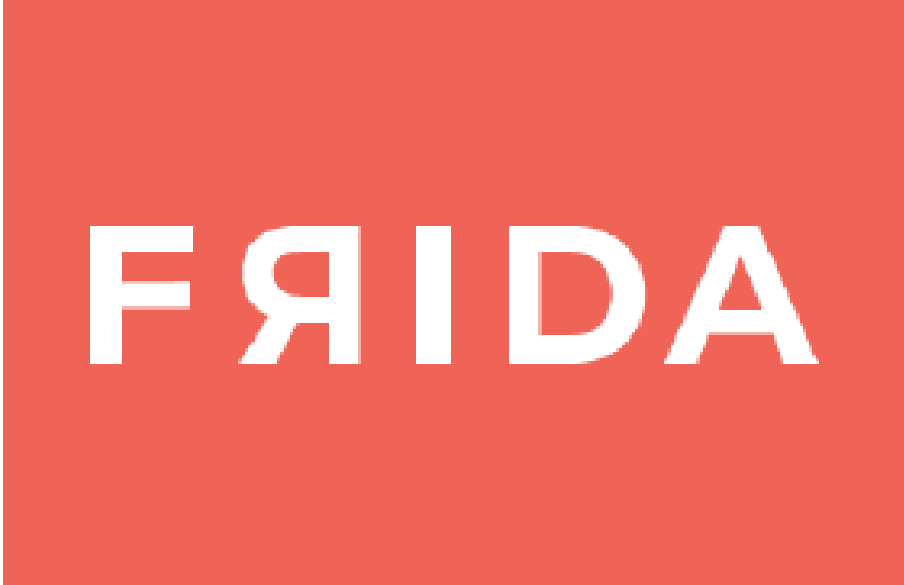
Reversing Tools - Frida Gadget

- uygulama içine gömülen (embed edilen) bir Frida bileşenidir
- frida-server gerektirmeden çalışabilir
- genellikle libfrida-gadget.so olarak APK içine eklenir
- uygulama başlarken otomatik olarak yüklenir
- TCP/IP üzerinden dışarıdan bağlantı kabul edebilir
- root gerektirmeden instrumentation yapılmasına olanak sağlar



Reversing Tools - Frida Trace

- hedef süreçteki fonksiyonları otomatik olarak trace etmek için kullanılan bir araçtır
- seçilen API'lere hızlıca hook ekleyip çağrılarını loglar
- frida-server/USB bağlantısı üzerinden çalışan bir CLI aracıdır
- hook scriptlerini otomatik üretir (manuel yazım gerektirmez)
- daha çok hızlı analiz ve keşif (recon) amacıyla kullanılır

The logo for Frida, featuring the word "FRIDA" in a bold, white, sans-serif font. The letters are slightly shadowed, giving them a 3D appearance as if they are floating above or attached to a red rectangular background.

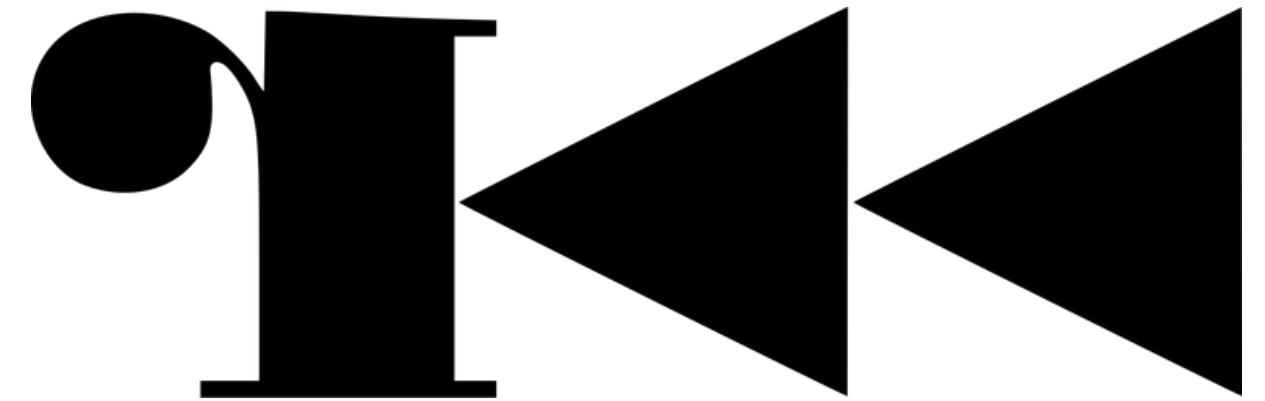
Reversing Tools - Ghidra

- Ghidra açık kaynaklı bir reverse engineering aracıdır
- derlenmiş binary'leri decompile ederek okunabilir koda çevirir
- statik analiz için kullanılır (çalıştırmadan inceleme)
- Java tabanlıdır ve çok platformludur
- NSA tarafından geliştirilmiştir



Reversing Tools - Radare2

- radare2 açık kaynaklı bir reverse engineering framework'üdür
- hem statik hem dinamik analiz yapabilir
- komut satırı tabanlıdır (CLI)
- düşük seviyeli binary inceleme ve patch işlemleri sağlar
- script edilebilir yapıya sahiptir
- farklı amaçlar için geliştirilmiş bir çok plugini mevcuttur.



Android Security Model

Detection Points
(real-life RASP usecases)

Detection Points - Root Detection

- cihazın root'lu olup olmadığını tespit etmeye çalışır
- sistem dosyaları (/system/xbin/su vb.) ve binary'leri kontrol eder
- root yönetim uygulamalarını (SuperSU, Magisk) arar
- sistem izinleri ve mount durumlarını analiz eder
- **amaç:** uygulamanın güvenli olmayan ortamda çalışmasını engellemek

Detection Points - SSL Pinning

- uygulamanın yalnızca belirli SSL sertifikalarına güvenmesini sağlar
- sunucunun sertifikası veya public key'i uygulama içine gömülür
- bağlantı sırasında gelen sertifika bu değerle karşılaştırılır
- uyuşmazlık varsa bağlantı reddedilir
- MITM (Man-in-the-Middle) saldırılarını engellemeyi amaçlar
- **amaç:** ağ trafiğinin izlenmesini ve değiştirilmesini önlemek

Detection Points - Debugger Detection (Frida,LLDB/GDB)

- uygulamaya bir debugger bağlanıp bağlanmadığını tespit eder
- ptrace, TracerPid gibi mekanizmaları kontrol eder
- debug flag ve runtime durumlarını inceler
- breakpoint / step işlemlerini dolaylı olarak algılayabilir
- **amaç:** tersine mühendislik ve analiz sürecini zorlaştırmak

Detection Points - CRC / Hash Check

- belirli method'ların bytecode/native içeriğinin hash/CRC değeri hesaplanır
- uygulama çalışırken aynı method tekrar hesaplanıp karşılaştırılır
- method hooklandığında (örn. inline patch, trampoline, bytecode değişimi) değer değişir
- beklenen CRC ile uyuşmazsa müdahale tespit edilir
- genelde kritik fonksiyonlar (auth, crypto, check'ler) hedef alınır
- **amaç:** kritik kodlara yapılan müdahaleyi (hook/patch) tespit etmek

Detection Points - Tampering Detection

- uygulamanın genel olarak değiştirilip değiştirilmediğini kontrol eder
- APK imzası, dosya hash'leri, resource ve manifest kontrol edilir
- repack, patch veya yeniden imzalama durumlarını tespit eder
- bazen runtime'da loaded code / memory değişiklikleri de incelenir
- **amaç:** uygulamanın orijinal hali dışında çalışmasını engellemek

Android Security Model

let's knock them out!